

Online Planning Monte-Carlo Algorithm with UCT for Partially Observable Markov Decision Processes Problems

Wentao Yang

Advisor: Mihai Anitescu

Approved _____

Date _____

November, 2025

Abstract

Partially Observable Markov Decision Processes (POMDPs) provide a framework for modeling decision-making under uncertain information observed from the interaction environment, which is often the case in the real world. However, solving them is computationally challenging due to the need to maintain probability distributions over observations, transitions, and beliefs in a potentially exponentially large state space. This paper studies the application of an online planning algorithm, Partially Observable Monte Carlo Planning (POMCP) with Upper Confidence bounds for Tree (UCT), converts it into a history MDP problem, proves its convergence, and implements it on the classical Rock Sample problem.

Contents

1	Introduction	3
2	Formal Problem Definitions	4
2.1	Markov Decision Process (MDP)	4
2.2	Partially Observable Markov Decision Process (POMDP)	4
2.3	Belief update	5
2.4	Policies and value functions	5
2.5	γ Selection and Undiscount problem	6
3	RockSample POMDP Problem	7
3.1	Problem Formulation	7
3.2	Belief State (b)	9
4	POMCP Algorithm: Monte Carlo Approach Planning	9
4.1	The Agent-Environment Interaction Loop	10
4.2	The Planner's Internal Process: Partially Observable UCT (PO-UCT)	10
5	Pseudocode of the POMCP Algorithm	12
6	Convergence of POMCP	15
6.1	Lemma 1: Equivalence of Value Functions	15
6.2	Lemma 2: Equivalence of Rollout Distributions	17
6.3	Theorem 1: Convergence of PO-UCT	18
7	Bandit-Based Monte-Carlo Planning	19
8	Experiment	26
8.1	Convergence of the Value Estimate $V(h)$	26
8.1.1	Mean Value Convergence	26
8.1.2	Variance Converges Zero	27
8.2	Convergence of Policy Quality	27

9	Future work and Discussion	28
9.1	The MCMC Sampling Problem as a POMDP	28
9.2	Accelerating Sampling via Online Planning	30
9.3	Challenges of Computational Trade-offs	30
10	Conclusions	30

1 Introduction

Sequential decision-making under uncertainty is a critical challenge in artificial intelligence. The Markov Decision Process (MDP) framework provides a robust mathematical foundation for this problem, modeling the interaction between an agent and a stochastic, fully observable environment [2, 16]. However, the core assumption of full observability is often violated in real-world applications [24]. In practical cases, such as robotic navigation [19], medical diagnosis [25], or spoken dialogue systems [30, 28], agents can only operate with limited, noisy, or ambiguous sensor data.

The Partially Observable Markov Decision Process (POMDP) framework was developed to provide a principled mathematical model for these problems [1, 7]. In a POMDP, the agent cannot access the true environment state s . Instead, it receives an observation o , which provides only partial information. Therefore, the agent must maintain a belief state $b(s)$, which is a probability distribution over all possible hidden states, given the entire history of past actions and observations [24, 4]. This process effectively transforms the original POMDP into a fully observable MDP defined over a continuous state space—the belief simplex. While this method sounds elegant, solving this belief-state MDP is still difficult. The problem is known to be PSPACE-complete [14]. The continuous nature of the belief space causes the curse of dimensionality, and the exponential growth of possible action-observation histories leads to the curse of history as discussed in [20].

Early attempts to solve POMDPs focused on offline methods that compute a complete policy before execution. Exact value iteration algorithms are among the most famous, which compute a piecewise-linear and convex value function, and are computationally feasible for small POMDPs [23, 24]. This encouraged the development of approximate offline methods, the most notably is Point-Based Value Iteration (PBVI) [15]. PBVI and its many successors, such as Heuristic Search Value Iteration (HSVI) [22] and SARSOP [9], gain scalability by computing a policy over only a finite set of reachable belief points.

A different paradigm is online planning, which avoids the costly computation of a global policy. Instead, an online planner focuses all its computational effort on finding the best action from the agent’s current belief state, just before an action is taken. [17] shows that the online method is a viable alternative for solving large POMDP problems. This approach was revolutionized in fully observable domains by Monte-Carlo Tree Search (MCTS) [5], particularly through the Upper Confidence bounds for Trees (UCT) algorithm [8]. UCT balances exploration and exploitation, combining the scalability of random sampling with the guidance of a search tree, and led to breakthrough performance in challenging games like Go [21].

In this paper, we study the application of Partially Observable Monte Carlo Planning (POMCP), introduced in [20], one of the online planning algorithms, which also serves as the foundation for modern extensions like rule-based [12] and risk-aware shielding [13]. This framework effectively adapts the MCTS and UCT algorithms to the POMDP setting by incorporating a particle belief update, as introduced in [27]. We first formally convert the POMDP into a history-based MDP, which provides the theoretical foundation for proving the algorithm’s convergence. We then implement this algorithm and validate it on the classic RockSample problem, a high-dimensional benchmark.

2 Formal Problem Definitions

2.1 Markov Decision Process (MDP)

MDP is a mathematical framework for modeling decision-making in a *fully observable* environment, where an agent's current state s is always completely known. An MDP is defined by a 5-tuple (S, A, T, R, γ) , where:

- S : A set of states s .
- A : A set of actions a .
- T : The transition model, $T : S \times A \rightarrow S$, $P(s'|s, a) = P(s_{t+1} = s' | s_t = s, a_t = a)$, which defines the probability of transitioning from state s to state s' after taking action a .
- R : The reward function, $R : S \times A \rightarrow \mathbb{R}$, $R_t = R(s_t, a_t)$, which provides the immediate reward R_t received after taking action a_t in state s_t .
- γ : The discount factor, $\gamma \in (0, 1]$.

At each time step t , the agent observes its state $s_t \in S$, chooses an action $a_t \in A$, transitions to a new state s_{t+1} according to $P(s_{t+1}|s_t, a_t)$, and receives a reward $R_t = R(s_t, a_t)$. As in an MDP, the goal is to find a policy $\pi(a|s)$ that maximizes the expected future discounted reward:

$$E \left[\sum_{t=0}^h \gamma^t R_t \right]$$

h is the planning horizon set by the planner how far it will explore.

2.2 Partially Observable Markov Decision Process (POMDP)

What distinguishes a POMDP from a fully observable MDP is that the agent now perceives an observation $o \in O$, instead of observing s_{t+1} directly. The discrete set of observations $O = \{o_1, o_2, \dots, o_M\}$ represents all possible sensor readings the agent can receive. Which observation the agent receives depends on the state s and may also be conditional on its action a , and is drawn according to the observation function $Z : S \times A \times O \rightarrow [0, 1]$. The probability of observing o in state s after executing a is $Z(s, a, o)$. For Z to be a valid probability distribution over possible observations, it is required that $\forall s \in S, a \in A, o \in O, Z(s, a, o) \geq 0$ and that $\sum_{o \in O} Z(s, a, o) = 1$. Alternatively, the observation function can also be defined as $Z : S \times O \rightarrow [0, 1]$ reflecting domains in which the observation is independent of the last action in some problem settings.

Definition A *partially observable Markov decision process* is a tuple $\langle S, A, O, T, Z, R \rangle$ in which S is a finite set of states, A is a finite set of actions, O is a finite set of observations, T is a *transition function* defined as $T : S \times A \times S \rightarrow [0, 1]$, Z is an *observation function* defined as $Z : S \times A \times O \rightarrow [0, 1]$ and R is a *reward function* defined as $R : S \times A \times S \rightarrow \mathbb{R}$

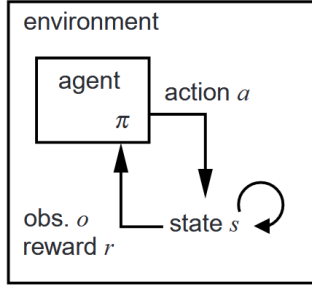


Figure 1: This figure reproduce from [24] Fig1. POMDP agent interacting with its environment and the paper introduces continuous and structured representations POMDP

2.3 Belief update

The agent needs a large amount of memory when considering optimal decision-making in a POMDP. A straightforward implementation of memory would be to store the sequence of actions executed and observations received. However, such a simple form of memory can grow indefinitely over time, making it impractical for long planning horizons. Fortunately, a better option exists, as we can transform the POMDP to a belief-state MDP in which the agent summarizes all information about its past using a belief $b(s)$. This transformation requires that the transition and observation functions are known to the agent.

Since the true state s is unknown, a POMDP agent must maintain a probability distribution over the entire state space S . This distribution, called $b(s)$, is known as the belief state that could be the form of vectors or a cloud of particles. It represents the agent's belief of $P(s_t = s | \text{history})$. After having taken the action a and observing o , an agent needs to update its belief in the state the environment may be in. Since the state is Markovian, maintaining a belief over the states solely requires knowledge of the previous belief state, the action a taken, and the current observation o . The update is denoted

$$b^{ao}(s') = \frac{p(o|s', a)}{p(o|b, a)} \sum_{s \in S} p(s'|s, a) b(s)$$

where $p(s'|s, a)$ and $p(o|s', a)$ are defined by probability from T and Z respectively.

$$p(o|b, a) = \sum_{s' \in S} p(o|s', a) \sum_{s \in S} p(s'|s, a) b(s)$$

is a normalizing constant.

This belief update step allows the agent to integrate new information and maintain a coherent understanding of its uncertainty as it moves through the environment.

2.4 Policies and value functions

As in the fully observable MDP setting, the agent's goal is to choose actions that fulfill its task as well as possible while learning an optimal policy. In POMDPs, an optimal policy

$\pi^*(b)$ maps beliefs (the probability distribution based on history actions) to actions. Contrary to MDPs, the policy $\pi(b)$ is a function over a continuous set of probability distributions over $\mathcal{S}$, where $\mathcal{S}$ is the set of all possible true states of the environment.

Memoryless policies can either be probabilistic mappings, $\pi : h(\text{history}) \rightarrow \Delta(A)$, where $\Delta(A)$ represents the set of all possible probability distributions over the actions, or deterministic mappings $\pi : h(\text{history}) \rightarrow A$. Here, we will choose a probabilistic policy definition. So belief state is $b(s|h_t = h) = P(s_t = s | \text{history} = h_t)$, and we have a policy π for this to have an action a , it is the same as saying with history h_t , what is the probability we will choose action $a_t = a$. So we have $\pi(h, a) = P(a_t = a | h_t = h)$. A policy π can be characterized by a value function V^π , which is defined as the expected future discounted reward $V^\pi(b)$ the agent can gather by following π starting from belief b :

$$V^\pi(b) = \mathbb{E}_\pi \left[\sum_{t=0}^h \gamma^t R(b_t, \hat{\pi}(b_t)) \middle| b_0 = b \right],$$

where $\hat{\pi}(b_t)$ is a specific action the agent chooses with probability from the policy π 's probability distribution, so $R(b_t, \hat{\pi}(b_t)) = \sum_{s \in \mathcal{S}} R(s, \hat{\pi}(b_t)) b_t(s)$. A policy π which maximizes V^π is called an optimal policy π^* ; it specifies for each b the optimal action to execute at the current step, assuming the agent will also act optimally at future time steps. The value of an optimal policy π^* is defined by the optimal value function V^* . It satisfies the Bellman optimality equation.

$$V^*(b) = \max_{a \in \mathcal{A}} \left[\sum_{s \in \mathcal{S}} R(s, a) b(s) + \gamma \sum_{o \in \mathcal{O}} p(o|b, a) V^*(b^{ao}) \right]$$

with b^{ao} (2.3) and $p(o|b, a)$ (2.3) as defined before. When $V^*(b)$ holds for every b We are assured that the solution is optimal.

2.5 γ Selection and Undiscount problem

If we set the discount factor $\gamma = 1$, this may result in mathematical and algorithmic challenges, particularly regarding the Bellman equation and algorithm convergence.

In many POMDPs, the horizon is not finite $h \rightarrow \infty$. If $\gamma = 1$, the agent's goal is to maximize the total undiscounted sum $\sum_{t=0}^{\infty} R_t$. If the agent can enter any cycle of actions that yields positive rewards, the expected future reward $V(b)$ for that cycle becomes infinite.

This causes the Bellman optimality equation failure:

$$V^*(b) = \max_{a \in \mathcal{A}} \left[R(b, a) + 1 \cdot \sum_{o \in \mathcal{O}} p(o|b, a) V^*(b^{ao}) \right]$$

If all subsequent value functions $V^*(b^{ao})$ are ∞ , then we get $V^*(b) = \max_a [R(b, a) + \infty] = \infty$. The agent can no longer distinguish between the values of different actions, as all lead to an infinite expected return.

3 RockSample POMDP Problem

There are many collected POMDP problems here[26]. We take a typical POMDP rock sample problem as an example and experiment with it here, but we consider the case where moving will incur a cost, which is similar to [10] Cost-Constrained POMCP. Here, the agent (rover) starts from $(0, 0)$ of the square grid world $n * n$, and the exit is the last column plus one, that is, move west when the agent is at any cell of the last column. And the agent collects or checks k rocks in the world. The agent knows exactly the positions of rocks and itself position all the time.

The "partial observability" arises because the agent's sensors are insufficient to determine the complete state of the world, like the quality of the rock, forcing it to analyze actions and rewards under uncertainty. Moreover, the optimal action depends on information that is not directly observable. In RockSample, the decision to Sample depends critically on the hidden rocktypes variable. Since the agent must act based on incomplete information. Following is the Formal Problem Formulation. This POMDP tuple $(S, A, T, R, O, Z, \gamma)$ is explicitly defined by a series of classes. γ is the discount factor; we set it to 0.95, which makes the future reward converge to 0 not too fast.

3.1 Problem Formulation

State Space S . The state represents the complete, true configuration of the world. A state is defined by a combination of the agent's exact (x, y) position, a tuple $K = (r_1, r_2, \dots, r_k) \in \{0, 1\}^k$ containing the true quality Good or Bad for all k rocks, and a boolean flag indicating if the agent has exited. The total number of states is $|S| = (\text{Number of Positions}) \times 2^K \times 2$. The 2^K term causes an exponential explosion in the state space, making it intractable to enumerate.

$$s = (pos_{rover}, status_{rocks}, flag) = ((x, y), (r_1, r_2, \dots, r_k), t)$$

where $t \in \{\text{true}, \text{false}\}$.

Action Space A . This is the set of all possible actions the agent can perform. The actions can be classified into three types:

- Movement: North, South, East, West. These actions deterministically move the agent one cell on the grid.
- Sampling: collect the rock to have positive rewards or a bad penalty. The agent can only collect the rock at its current location. It can not collect any rocks that are not in the agent's position.
- Checking: $Check_i$ for each rock with label i . The agent uses its sensor to check the quality of a specific rock from its current position. It can check any rocks at any position.

Transition Model T . Let the next state be $s' = (pos', R', flag')$. $T(s'|s, a) = P(s_{t+1} = s'|s_t = s, a_t = a)$, defines how the state of the world changes in response to an agent’s action. In the RockSample problem, all transitions are deterministic, meaning $P(s'|s, a)$ is either 1 or 0. But for problems like Medical Diagnosis and Treatment problem [18]. and Machine Maintenance[3], which also use online planning as we will discuss later, these problems’ transition functions are probability distributions and need to be considered in the belief state update. But this is not the main part of POMDP, so we choose a simpler transition model. We define s' and $flag'$ to be the next state and flag in s' . The deterministic transition function is defined as follows:

$$s' = T(s, a) = \begin{cases} (pos, R, true) & \text{if } flag = true \\ (MOVE(pos, a), R, flag') & \text{if } a \in \text{Moves} \\ (pos, R, flag) & \text{if } a = \text{Sample or } a \in \text{Checks} \end{cases}$$

Reward Model R . The reward model provides the numerical signal that guides the agent’s planning. The goal is to maximize the cumulative reward. The reward structure is as follows:

- **Positive Rewards:**

- A reward of **+30** is given for sampling a ‘Good’ rock.
- A reward of **+30** is given for successfully moving into the exit area.

- **Negative Rewards (Penalties):**

- A penalty of **-10** is given for sampling a ‘Bad’ rock.

- **Action Costs:**

- A small cost of **-1** is applied to all move and check actions, as well as for attempting to sample an empty location. This incentivizes efficiency. While the traditional rocksample problem set this is 0, meaning no cost for move and check[22], they use (*HSVI*) Heuristic Search Value Iteration, which is widely used before online planning.

Observation Model Z and O . This model is the source of all uncertainty and partial observability. The set of possible observations O is {**ObservedGood**, **ObservedBad**, **Null**}. When the agent performs a *Check_i* action, it receives a noisy observation about rock i . The probability of this observation being correct depends on the distance to the rock—the closer the agent is, the more reliable the sensor. Here is the observation function Z with probability

$$P(\text{correct observation}) = 0.5 \times (1 + 2^{-d/\eta})$$

where η is a constant that controls how quickly the sensor’s accuracy degrades with distance. At a great distance, the accuracy approaches $\frac{1}{2}$. random guess. For any action other than Check, the agent receives a non-informative Null observation.

3.2 Belief State (b)

The core uncertainty lies in the qualities of the rocks. Therefore, the belief state primarily captures the probability of each possible configuration of rock qualities. For a problem with k rocks, there are 2^k such configurations.

Particles for Representation In online planning the idea like [11] particle filter belief is widely used to manage the belief state. Even if the agent always knows its exact position, representing the belief state as a dense probability vector over all $n^2 \times 2^k$ states is computationally intractable for even moderate values of k . The provided code addresses this issue by utilizing a particle filter. The belief b is approximated by a large set of N weighted samples, referred to as particles.

- A Particle is a State Hypothesis: Each particle is a single, complete, concrete hypothesis of the true world state. For example, one particle might be: $\text{State}(\text{position} = (1, 1), \text{rocktypes} = (\text{Good}, \text{Bad}, \text{Good}, \dots), \text{Flag} = \text{False})$
- The agent’s belief is the entire collection of these N particles. The probability of a certain rock configuration is approximated by the fraction of particles in the set that contain that configuration. And this is an initialized uniform distribution of rock qualities.

Belief Update (The Bayes Filter) The agent’s belief is updated after every interaction with the environment, taking an action a and receiving an observation o . This update process is a practical implementation of the Bayes’ filter equation just as (2.3)

In the particle filter implementation, this equation is approximated through a three-step process that occurs in the main *testplanner* loop after a real observation is received:

1. **Prediction:** For each particle in the current belief set, the agent simulates the action a . It passes the particle through the Transition model to generate a new particle, forming a new temporary set of predicted particles. This step corresponds to the summation term $\sum_{s \in S} P(s' | s, a) \cdot b(s)$.
2. **Weighting:** The agent takes the real observation o it received from the environment. For each particle in the predicted set, it calculates a weight, w , equal to the likelihood of that particle being correct, given the observation: $w = P(o | s'_{\text{particle}}, a)$. This is done using the Observation Model.
3. **Resampling:** The agent creates a new, final belief set by drawing N particles from the weighted, predicted set. Particles with higher weights are more likely to be selected and duplicated, while particles with very low weights are likely to be discarded. This new set of particles represents the updated belief b' .

4 POMCP Algorithm: Monte Carlo Approach Planning

Methods such as value iteration, Exact value iteration, and Point-based value iteration are well-introduced and discussed in [24]. It is useful for solving POMDPs with limited states,

like the Tiger problem. To solve the RockSample problem of this large dimension, we prefer to use an advanced online planning algorithm called POMCP (Partially Observable Monte Carlo Planning) [20]. This method does not compute a complete policy beforehand. Instead, it plans and does simulations from the agent’s current belief state to find the best immediate action. The core of POMCP is Monte Carlo Tree Search (MCTS), a simulation-based search technique.

To understand the process, we will distinguish between the high-level agent-environment interaction and the low-level planner’s internal simulations.

4.1 The Agent-Environment Interaction Loop

This is the outer loop that describes the agent’s lifecycle, as described in **Algorithm 1**. For each step of the episode, the following sequence occurs:

1. **Plan:** The agent calls the agent to determine the best action based on its current belief. This triggers the entire MCTS process described below.
2. **Act & Transition:** The environment transitions to a new true state based on the chosen action. This is handled by the transition model.
3. **Observe:** The agent receives a real observation o from the environment based on the new state and the action taken, via the call to the observation model, but we emphasize that this does not change the whole state s the agent is in.
4. **Update Belief & Planner:** The agent updates its internal history and belief based on the action and observation. Crucially, the planner itself is also updated with the real action and observation as shown in **Algorithm 2**.

4.2 The Planner’s Internal Process: Partially Observable UCT (PO-UCT)

This is the inner loop as shown in **Algorithm 3**, where the Monte Carlo algorithm is used. The fundamental difference from standard MCTS is that PO-UCT uses a search tree of histories (h) instead of states s . Each node in the tree $T(h)$ represents a specific action-observation history and stores two key values:

- $N(h)$: The number of times this history node has been visited during simulation.
- $V(h)$: The estimated value that is the mean return of all simulations that have passed through this history h .

Each simulation follows four distinct steps.

Step 1: Selection and Branch Pruning Algorithm starts at the root of the search tree and traverses down to find a promising area to explore. From [6], they analyze the mechanics behind Monte Carlo Tree Search about the selection mechanism that balances exploration vs. exploitation. And At each node h in the tree, an action is chosen according

to the UCB1 tree policy (Upper Confidence bounds for Trees) formula from UCT analysis [8], has proven to be reliable.

$$a^* = \arg \max_{a \in A(s)} \left(Q(s, a) + C \sqrt{\frac{\ln N(s)}{N(s, a)}} \right) \quad (1)$$

$Q(s, a)$: exploitation term. The current average value represents the rewards of taking action a from state s . It's calculated by dividing the total reward from simulations that chose this action by the number of times it has been chosen (V/N).

C : The exploration constant. A higher C value encourages more exploration of lesser-known actions, while a lower C value favors exploiting actions that are already known to be effective.

$N(s)$: The total times the parent node s has been visited during the entire search process.

$N(s, a)$: Times the specific action a has been chosen from state s .

Initially, the planner is at the root node s_0 . When $N(s) = 0$ and $N(s, a) = 0$ for all actions, the UCT algorithm handles this with a simple but effective rule: any action a that has not yet been tried is assigned an infinite exploration bonus. The simulation selection process is as follows.

The algorithm randomly picks one of these infinite-score actions. For example, let a_1 be the first action. A full simulation is run, which returns a total reward of R_1 . Then, the tree is updated. The statistics for the root node s are now: $N(s) = 1$, $N(s, a_1) = 1$, $Q(s, a_1) = R_1$, others are all 0. The planner is again at the root node s . It uses the UCT formula to choose a_2 randomly, then update to $N(s) = 2$, $N(s, a_2) = 1$, $Q(s, a_2) = R_2$.

This process repeats until every action from the root node is chosen at least once. And we can deduce that every action at state s will be explored, since if $N(s, a) = 0$, then exploration is infinite, meaning we will explore the action.

A branch of a bad represented action will quickly accumulate a low $Q(h, a)$ value. As its visit count $N(h, a)$ increases, the exploration term of the UCT formula shrinks, making its overall score very low. The algorithm will naturally stop selecting this unpromising branch in favor of branches with higher potential. This process can effectively prune the search by focusing computational effort on the most valuable paths.

Step 2: Expansion When the selection process reaches a leaf node h , the tree is expanded. A new child node $h' = h_{ao}$ is created after an action a_t is taken and a real observation o_{t+1} is received. The prune tree step performs the following optimization: It navigates from the root of the old tree T_t down the branch corresponding to the action a_t that was actually taken. From the action, it follows the branch corresponding to the observation o_{t+1} that was actually received. The node at the end of this path becomes the new root of the tree T_{t+1} . All other branches from the old root are pruned. This allows the planner to reuse all

the simulation work from the previous step that is still relevant to the agent’s new belief, making the planning process highly efficient.

Step 3: Simulation and Rollout This is the core Monte Carlo step. As described in **Algorithm 4**, the planner runs a fast simulation from the newly expanded node until a terminal state or maximum depth is reached. This simulation employs a simple default rollout policy to choose actions, and the total discounted reward, R , is recorded.

Improving the Rollout Policy: The simple random policy is a known performance bottleneck [29]. Its high-variance estimates can slow down the convergence of the main UCT algorithm, as suggested by experimental results showing unstable variance. An improvement might be to replace this random policy domain-specific **heuristic policy**. For RockSample, such a policy could be:

1. Move towards the nearest rock that is believed (based on the current belief state) to be ‘Good’.
2. If no rock is ‘Good’, move towards the nearest unchecked rock.
3. If all rocks are sampled or believed ‘Bad’, move towards the exit area.

This heuristic provides a more accurate, lower-variance estimate of a node’s value without the full cost of a nested MCTS search, leading to faster and more stable overall convergence.

Step 4: Backpropagation The total reward obtained from the rollout simulation is propagated back up the tree. The visit counts ($N(h)$ and $N(h, a)$) and value estimates ($Q(h, a)$) of all nodes and actions along the path taken during the selection step are updated with the new information.

Final Decision and Tree Reuse After running a large number of simulations, the planner has built a search tree that accurately estimates the values of the immediate actions. The planner chooses the action $a_t = a^*$ at the root node h_t that has the highest estimated value ($Q(h_t, a)$) as introduced (1).

5 Pseudocode of the POMCP Algorithm

We provide the pseudocode for the POMCP algorithm as it is applied to general large POMDP problems with high dimensions. The algorithm is split into two parts: the high-level agent-environment interaction loop and the detailed planner’s internal MCTS process. The simulation time is num_sims , and n_steps is the horizon, the number of actions we will do. max_depth is only for one-time simulation, specifying how deep it will search.

Algorithm 1 High-Level Agent-Environment Interaction Loop

```
1: Initialize agent's belief  $b_0$  with a set of  $N$  particles, planner with an empty search tree  $T_0$ 
2: for  $t = 0$  to  $n\_steps - 1$  do
3:                                      $\triangleright$  PLANNING STEP
4:    $a_t \leftarrow \text{POMCP\_PLAN}(b_t, T_t)$ 
5:                                      $\triangleright$  MCP EXECUTION STEP
6:   Receive real observation  $o_{t+1}$  and reward  $r_t$ 
7:                                      $\triangleright$  BELIEF UPDATE STEP
8:    $b_{t+1} \leftarrow \text{PARTICLE\_FILTER\_UPDATE}(b_t, a_t, o_{t+1})$ 
9:                                      $\triangleright$  TREE PRUNING (REUSE) STEP
10:   $T_{t+1} \leftarrow \text{PRUNE\_TREE}(T_t, a_t, o_{t+1})$ 
11: end for
```

Algorithm 2 $\text{PARTICLE_FILTER_UPDATE}(b_t, a_t, o_{t+1})$

Require: Current belief b_t (a set of N particles $S_t = \{s_t^{(1)}, \dots, s_t^{(N)}\}$)
Require: Action a_t taken
Require: Real observation o_{t+1} received
Ensure: New belief b_{t+1} (a new set of N particles S_{t+1})

```
1:  $B_{pred} \leftarrow \emptyset, W \leftarrow \emptyset, S_{t+1} \leftarrow \emptyset, TotalWeight \leftarrow 0$   $\triangleright$  Initialize temporary set for predicted particles, temporary list for weights, the new particle set (output)
2:                                      $\triangleright$  1. PREDICTION STEP
3: for each particle  $s_t^{(i)}$  in  $b_t$  do
4:    $s_{t+1}^{(i)} \leftarrow T(s_{t+1}^{(i)} | s_t^{(i)}, a_t)$   $\triangleright$  Sample next state using Transition Model
5:   Add  $s_{t+1}^{(i)}$  to  $B_{pred}$ 
6: end for
7:                                      $\triangleright$  2. WEIGHTING STEP
8: for each predicted particle  $s_{t+1}^{(i)}$  in  $B_{pred}$  do
9:    $w_i \leftarrow Z(o_{t+1} | s_{t+1}^{(i)}, a_t)$   $\triangleright$  Get likelihood from Observation Model
10:  Add  $w_i$  to  $W$ 
11:   $TotalWeight \leftarrow TotalWeight + w_i$ 
12: end for
13:                                      $\triangleright$  3. RESAMPLING STEP
14: if  $TotalWeight = 0$  then
15:    $\text{HANDLE\_PARTICLE\_DEPRIVATION}$   $\triangleright$  re-initialize from  $b_0$ 
16:   return  $b_0$ 
17: end if
18: for  $i = 1$  to  $N$  do
19:   Draw a particle  $s_{t+1}^{(j)}$  from  $B_{pred}$  with probability  $\propto w_j / TotalWeight$ 
20:   Add  $s_{t+1}^{(j)}$  to  $S_{t+1}$   $\triangleright$  High-weight particles are duplicated, low-weight are discarded
21: end for
22:  $b_{t+1} \leftarrow S_{t+1}$ 
23: return  $b_{t+1}$ 
```

Algorithm 3 Planner's Internal Process: POMCP_PLAN(b, T)

Require: Current belief b (a set of particles), current search tree T

- 1: **for** $i = 1$ **to** num_sims **do**
 - 2: Sample a state $s \sim b$ ▷ Randomly pick one particle from the belief set
 - 3: SIMULATE($s, T.root, 0$) ▷ Run one full simulation from the root of the tree
 - 4: **end for**
 - 5: **return** The action a from the root with the highest value, $\arg \max_a Q(T.root, a)$
-

Algorithm 4 SIMULATE($s, h, depth$)

Require: Current state s , tree node h , current depth $depth$

- 1: **if** $depth \geq max_depth$ **then**
 - 2: **return** 0
 - 3: **end if**
 - 4: **if** h is not a leaf node in T **then** ▷ **Step 1: SELECTION**
 - 5: $a \leftarrow \arg \max_{a'} \left(Q(h, a') + C \sqrt{\frac{\ln N(h)}{N(h, a')}} \right)$ ▷ Select action using UCT formula
 - 6: $(s', o, r) \leftarrow \text{GENERATE}(s, a)$ ▷ Use Transition, Observation, and Reward models,
Generate function $G(s, a) \rightarrow (T(s, a), Z(s, a), R(s, a)) = (s', o, r)$
 - 7: $h' \leftarrow \text{GET_CHILD}(h, a, o)$ ▷ Find or create the next history node
 - 8: $R \leftarrow r + \gamma \cdot \text{SIMULATE}(s', h', depth + 1)$
 - 9: **else** ▷ **Step 2 & 3: EXPANSION & SIMULATION (ROLLOUT)**
 - 10: Expand node h by adding children for all valid actions
 - 11: $R \leftarrow \text{ROLLOUT}(s, depth)$ ▷ Perform a rollout from the current state
 - 12: **end if**
 - 13: ▷ **Step 4: BACKPROPAGATION**
 - 14: $N(h) \leftarrow N(h) + 1$
 - 15: **if** h is not the root node **then**
 - 16: $N(h_{parent}, a_{leading_to_h}) \leftarrow N(h_{parent}, a_{leading_to_h}) + 1$
 - 17: $Q(h_{parent}, a_{leading_to_h}) \leftarrow Q(h_{parent}, a_{leading_to_h}) + \frac{R - Q(h_{parent}, a_{leading_to_h})}{N(h_{parent}, a_{leading_to_h})}$
 - 18: **end if**
 - 19: **return** R
-

Algorithm 5 ROLLOUT($s, depth$)

Require: Current state s , current depth $depth$

- 1: **if** $depth \geq max_depth$ **then**
 - 2: **return** 0
 - 3: **end if**
 - 4: $a \leftarrow$ action from a simple rollout policy
 - 5: $(s', o, r) \leftarrow \text{GENERATE}(s, a)$
 - 6: **return** $r + \gamma \cdot \text{ROLLOUT}(s', depth + 1)$
-

6 Convergence of POMCP

The idea of casting a POMDP as a belief MDP is introduced and discussed in [4]. However, applying the UCT formula directly to large POMDPs, such as rocksample, is very computationally expensive. This happens if we turn every history or belief state into an MDP state, as the total number of states grows too large. POMCP offers a more efficient way. It applies UCT search to a history-based MDP, but it uses a state-based simulator. This simulator samples states from the current belief, which allows the algorithm to avoid handling the full belief state representation. Then we need to prove that POMCP converges to the optimal value function. We will show this for POMDPs with a finite horizon T , assuming we are given the true belief state $\mathcal{B}(s, h)$ at the start.

6.1 Lemma 1: Equivalence of Value Functions

We reproduce the statement of Lemma 1 in [20].

Lemma 1 (Lemma 1 in [20]). *Given a POMDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{Z})$, consider the derived MDP with histories as states, $\tilde{\mathcal{M}} = (\mathcal{H}, \mathcal{A}, \tilde{\mathcal{P}}, \tilde{\mathcal{R}})$, where the derived transition function $\tilde{\mathcal{P}}$ and reward function $\tilde{\mathcal{R}}$ are defined as:*

$$\tilde{\mathcal{P}}_{h, h_{ao}}^a = \sum_{s \in \mathcal{S}} \sum_{s' \in \mathcal{S}} \mathcal{B}(s, h) \mathcal{P}_{ss'}^a \mathcal{Z}_{s'o}^a \quad (2)$$

$$\tilde{\mathcal{R}}_h^a = \sum_{s \in \mathcal{S}} \mathcal{B}(s, h) \mathcal{R}_s^a \quad (3)$$

Then the value function $\tilde{V}^\pi(h)$ of the derived MDP is equal to the value function $V^\pi(h)$ of the POMDP, $\forall h \tilde{V}^\pi(h) = V^\pi(h)$.

To understand this lemma, we must first explain all its components:

- $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{Z})$: The original POMDP model. $\mathcal{S}$ state space $\mathcal{A}$ action space.
- $\mathcal{P}_{ss'}^a = P(s'|s, a)$: Probability of transitioning from state s to state s' given action a .
- $\mathcal{R}_s^a = \mathbb{E}[r_{t+1}|s_t = s, a_t = a]$: The expected reward for taking action a in state s .
- $\mathcal{Z}_{s'o}^a = P(o|s', a)$: Probability of receiving observation o given action a led to state s' .
- h : A history of a sequence of actions and observations $h = \{a_1, o_1, \dots, a_t, o_t\}$.
- $\mathcal{B}(s, h) = P(s_t = s|h_t = h) = b(s|h_t = h)$: The belief state, the probability that the true current state is s given the history h .
- $\tilde{\mathcal{M}}$: The derived MDP where each history $h \in \mathcal{H}$ is treated as a distinct state.
- $\tilde{\mathcal{P}}_{h, h_{ao}}^a$: The probability in the derived MDP of transitioning from history state h to history state h_{ao} by taking action a and observing o .

- $\tilde{\mathcal{R}}_h^a$: The expected reward in the MDP for taking action a from history state h .
- $V^\pi(h)$: The expected future return value in the original POMDP from history h when following policy π .
- $\tilde{V}^\pi(h)$: The expected future return value in the derived MDP from history state h when following policy π .

Proof of Lemma 1

Proof. We argue following the idea given in [20]. The Bellman equation for the value function $V^\pi(h)$ in the POMDP $V^\pi(h)$ is the expected return from history h following policy $\pi(h, a)$. This can be expanded by summing over all possible actions, states, next states, and observations:

$$V^\pi(h) = \sum_{s,a} \mathcal{B}(s, h) \pi(h, a) R_s^a + \sum_{s,a,s',o} \mathcal{B}(s, h) \pi(h, a) \gamma P_{ss'}^a Z_{s'o}^a V^\pi(h_{ao})$$

The first term is the instant reward, and the second term is the future reward. We can rearrange the terms:

$$V^\pi(h) = \sum_{a \in \mathcal{A}} \pi(h, a) \left[\left(\sum_{s \in \mathcal{S}} \mathcal{B}(s, h) \mathcal{R}_s^a \right) + \gamma \sum_{o \in \mathcal{O}} \left(\sum_{s \in \mathcal{S}} \sum_{s' \in \mathcal{S}} \mathcal{B}(s, h) \mathcal{P}_{ss'}^a \mathcal{Z}_{s'o}^a \right) V^\pi(h_{ao}) \right]$$

Now, we substitute the definitions of $\tilde{\mathcal{R}}_h^a$ and $\tilde{\mathcal{P}}_{h,h_{ao}}^a$ from the lemma statement:

$$V^\pi(h) = \sum_{a \in \mathcal{A}} \pi(h, a) \left[\tilde{\mathcal{R}}_h^a + \gamma \sum_{o \in \mathcal{O}} \tilde{\mathcal{P}}_{h,h_{ao}}^a V^\pi(h_{ao}) \right] \quad (4)$$

1. Base Case (Horizon $t = T$) At the final time step T , there is no future reward, so the value function $V^\pi(h_T)$ only consists of the expected immediate reward.

$$V^\pi(h_T) = \sum_{a \in \mathcal{A}} \pi(h_T, a) \tilde{\mathcal{R}}_{h_T}^a$$

For the derived MDP, the value function $\tilde{V}^\pi(h_T)$ is, by definition, the expected immediate reward:

$$\tilde{V}^\pi(h_T) = \sum_{a \in \mathcal{A}} \pi(h_T, a) \tilde{\mathcal{R}}_{h_T}^a$$

Therefore, at the horizon T , $V^\pi(h_T) = \tilde{V}^\pi(h_T)$. The base case holds.

2. Inductive Step Assume that for any history h_{t+1} at time $t + 1$, the hypothesis $V^\pi(h_{t+1}) = \tilde{V}^\pi(h_{t+1})$ holds. We will now prove it also holds for any history h_t at time t . From (4)

$$V^\pi(h_t) = \sum_{a \in \mathcal{A}} \pi(h_t, a) \left[\tilde{\mathcal{R}}_{h_t}^a + \gamma \sum_{o \in \mathcal{O}} \tilde{\mathcal{P}}_{h_t, h_t a o}^a V^\pi(h_t a o) \right]$$

The history $h_t a o$ is a history at time $t + 1$. Therefore, we can apply our inductive hypothesis, replacing $V^\pi(h_t a o)$ with $\tilde{V}^\pi(h_t a o)$:

$$V^\pi(h_t) = \sum_{a \in \mathcal{A}} \pi(h_t, a) \left(\tilde{\mathcal{R}}_{h_t}^a + \gamma \sum_{o \in \mathcal{O}} \tilde{\mathcal{P}}_{h_t, h_t a o}^a \tilde{V}^\pi(h_t a o) \right)$$

The right-hand side is precisely the definition of the Bellman equation for the value function $\tilde{V}^\pi(h_t)$ in the derived MDP $\tilde{\mathcal{M}}$. Therefore, $V^\pi(h_t) = \tilde{V}^\pi(h_t)$. □

6.2 Lemma 2: Equivalence of Rollout Distributions

$\mathcal{D}^\pi(h_T)$ (**POMDP rollout distribution**) The distribution of future histories generated by the practical simulation process used by the PO-UCT algorithm. Starting from the current history h_t with steps like

1. Sample an initial state $s_t \sim \mathcal{B}(s, h_t)$ from the current belief
2. Repeatedly sample actions a from the rollout policy $\pi(h, a)$.
3. Repeatedly sample the next state s' , observation o , and reward r using the original POMDP model $\mathcal{M}$

$\tilde{\mathcal{D}}^\pi(h_T)$ (**Derived MDP rollout distribution**) The distribution of future histories generated by the theoretical simulation process in the abstract derived MDP:

1. Start directly from the abstract history state h_t .
2. Repeatedly sample actions a from the same rollout policy $\pi(h, a)$.
3. Repeatedly sample the next history state h_{ao} (which is determined by sampling the observation o) and reward $\tilde{r}$ using the derived MDP model $\tilde{\mathcal{M}}$.

Similarly, We reproduce the statement of Lemma 2 in [20].

Lemma 2 (Lemma 2 in [20]). *For any rollout policy π , the POMDP rollout distribution is equal to the derived MDP rollout distribution.*

$$\forall \pi \quad \mathcal{D}^\pi(h_T) = \tilde{\mathcal{D}}^\pi(h_T)$$

We argue following the forward induction of policy distribution [20].

Proof of Lemma 2

Proof. At beginning, we assume the distributions are equal up to history h , $\mathcal{D}^\pi(h) = \tilde{\mathcal{D}}^\pi(h)$ and prove that they must also be equal for the next history h_{ao} .

1. Probability of h_{ao} via POMDP Rollout ($\mathcal{D}^\pi$) The probability of arriving at history h_{ao} is the probability of being at h , times the probability of choosing action a (given by π), times the total probability of then observing o . This total probability of observing o is found by summing over all possible underlying state transitions ($s \rightarrow s'$) that could produce it, weighted by the current belief $\mathcal{B}(s, h)$.

$$\begin{aligned}\mathcal{D}^\pi(h_{ao}) &= \mathcal{D}^\pi(h) \cdot \pi(h, a) \cdot P(o|h, a) \\ &= \mathcal{D}^\pi(h) \cdot \pi(h, a) \cdot \left(\sum_{s \in \mathcal{S}} \sum_{s' \in \mathcal{S}} \mathcal{B}(s, h) \mathcal{P}_{ss'}^a \mathcal{Z}_{s'o}^a \right)\end{aligned}$$

2. Probability of h_{ao} via Derived MDP Rollout ($\tilde{\mathcal{D}}^\pi$) The probability of transitioning from the "history state" h to the "history state" h_{ao} is given directly by the dynamics of the derived MDP $\tilde{\mathcal{M}}$:

$$\tilde{\mathcal{D}}^\pi(h_{ao}) = \tilde{\mathcal{D}}^\pi(h) \cdot \pi(h, a) \cdot \tilde{\mathcal{P}}_{h, h_{ao}}^a$$

By our inductive hypothesis $\mathcal{D}^\pi(h) = \tilde{\mathcal{D}}^\pi(h)$. Furthermore, from Lemma 1

$$\tilde{\mathcal{P}}_{h, h_{ao}}^a = \sum_{s \in \mathcal{S}} \sum_{s' \in \mathcal{S}} \mathcal{B}(s, h) \mathcal{P}_{ss'}^a \mathcal{Z}_{s'o}^a$$

By substituting this into the equation for $\tilde{\mathcal{D}}^\pi(h_{ao})$, both probability calculations become identical.

$$\tilde{\mathcal{D}}^\pi(h) \cdot \pi(h, a) \cdot \tilde{\mathcal{P}}_{h, h_{ao}}^a = \mathcal{D}^\pi(h) \cdot \pi(h, a) \cdot \left(\sum_{s \in \mathcal{S}} \sum_{s' \in \mathcal{S}} \mathcal{B}(s, h) \mathcal{P}_{ss'}^a \mathcal{Z}_{s'o}^a \right)$$

Therefore, $\mathcal{D}^\pi(h_{ao}) = \tilde{\mathcal{D}}^\pi(h_{ao})$. Since the probability of any single-step transition is identical in both models, the resulting distributions over whole histories h_T are identical. $\square$

6.3 Theorem 1: Convergence of PO-UCT

Again, we reproduce the statement of Theorem 1 in [20].

Theorem 1 (Theorem 1 in [20]). *For a suitable choice of C , the value function constructed by PO-UCT converges in probability to the optimal value function, $V(h) \xrightarrow{P} V^*(h)$, for all histories h that are prefixed by h_t . As the number of visits $N(h)$ approaches infinity, the bias of the value function, $\mathbb{E}[V(h) - V^*(h)]$ is $O(\log N(h)/N(h))$. Where C is the exploration constant used in the UCT formula from (1).*

Proof of Theorem 1

Proof. From **Lemma 1**, it is proved that our POMDP problem (with value function $V^\pi(h)$) is mathematically equivalent to a derived MDP. And **Lemma 2** proved that the simulation process our PO-UCT algorithm actually runs is statistically identical to a simulation process

in that derived MDP. Then we know running PO-UCT on the POMDP is equivalent to running the standard UCT algorithm on the derived MDP. PO-UCT inherits all the convergence properties of standard UCT, including the $O(\log N(h)/N(h))$ bias convergence rate from [8]. We give the detailed proof of the convergence rate in the next section. $\square$

7 Bandit-Based Monte-Carlo Planning

PO-UCT inherits the theoretical guarantees of the standard UCT algorithm, the $O(\log n/n)$ convergence rate of the estimation bias. To prove this convergence, we will address the inherent non-stationarity to tree-based Monte-Carlo planning.

During the search process, the value estimates of child nodes are continuously updated. Consequently, each internal parent node faces a multi-armed bandit problem where the underlying payoff distributions of its arms are drifting over time. To prove that the root node's estimate eventually converges to the true optimal value, we must show that this drift is controlled and that the algorithm's exploration mechanism remains robust.

In this section, we reconstruct and analyze the theoretical framework originally introduced by [8]. We present a sequence of theorems that systematically build up to the final convergence proof.

Theorem 2. *Consider UCB1 (1) applied to a non-stationary problem. Let $T_i(n)$ denote the number of plays of arm i . Then if i is the index of a suboptimal arm, and $n > K$, then:*

$$\mathbb{E}[T_i(n)] \leq \frac{16C_p^2 \ln n}{(\Delta_i/2)^2} + 2N_0 + \frac{\pi^2}{3}$$

Proof. At any time step t , we select a suboptimal arm i instead of the optimal arm $*$ only if the Upper Confidence Bound (UCB) of arm i exceeds that of arm $*$. Let $\bar{X}_{i,T_i(t-1)}$ be the empirical mean of arm i after $T_i(t-1)$ pulls, and let $c_{t-1,T_i(t-1)}$ be the exploration bonus. The condition for error is:

$$\bar{X}_{i,T_i(t-1)} + c_{t-1,T_i(t-1)} \geq \bar{X}_{*,T_*(t-1)} + c_{t-1,T_*(t-1)} \quad (5)$$

We define the exploration bonus as $c_{t,s} = 2C_p \sqrt{\frac{\ln t}{s}}$. The error condition implies that at least one of the following three specific failure events has occurred. If none of them had occurred, the optimal arm would have been chosen.

The first event is that the optimal arm $*$ is underestimated. The empirical mean $\bar{X}_{*,s}$ falls below its current true expected value $\mu_{*,s}$ minus the exploration bonus:

$$\bar{X}_{*,s} \leq \mu_{*,s} - c_{t,s} \quad (6)$$

The second event is that the suboptimal arm i is overestimated. Its empirical mean $\bar{X}_{i,s}$ exceeds its current true expected value $\mu_{i,s}$ plus the exploration bonus:

$$\bar{X}_{i,s} \geq \mu_{i,s} + c_{t,s} \quad (7)$$

The third event is that the confidence intervals are not tight enough to separate the arms, even if the estimates are accurate. The true mean of the optimal arm μ^* is less than the true mean of the suboptimal arm μ_i plus twice the confidence width.

$$\mu^* < \mu_i + 2c_{t,s} \quad (8)$$

In a stationary bandit, the third event is resolved by sufficient sampling. In the non-stationary case, we account for the drift $\delta_{i,s} = \mu_{i,s} - \mu_i$. We substitute the current non-stationary means $\mu_{*,s}$ and $\mu_{i,s}$ into the gap inequality. We assume the existence of a threshold N_0 such that for samples $s > N_0$, the exploration bonus dominates the drift, $c_{t,s} \geq 2|\delta_{i,s}|$.

Substituting the drift terms into the gap definition $\Delta_i = \mu^* - \mu_i$, the number of pulls $T_i(n)$ to be large enough such that

$$\Delta_i > 2c_{t,s} + |\delta_{i,s}| + |\delta_{*,T_*}| \quad (9)$$

Using the drift condition assumption for $s > N_0$, we can bound the right hand side by $3c_{t,s}$ or effectively $2c_{t,s}$. To avoid the third event, we need the confidence interval to be smaller than half the gap. Solving $c_{t,s} \approx \Delta_i/2$ for s :

$$2C_p \sqrt{\frac{\ln t}{s}} < \frac{\Delta_i}{2} \implies s > \frac{16C_p^2 \ln t}{\Delta_i^2} \quad (10)$$

Let $u = \lceil \frac{16C_p^2 \ln n}{\Delta_i^2} \rceil$. If the arm i is pulled more than u times, then the third case is impossible.

For the first two events, estimation errors, we need concentration inequalities, Hoeffding's Inequality.

$$\mathbb{P}(\bar{X}_{*,s} \leq \mu_{*,s} - c_{t,s}) \leq t^{-4} \quad \text{and} \quad \mathbb{P}(\bar{X}_{i,s} \geq \mu_{i,s} + c_{t,s}) \leq t^{-4} \quad (11)$$

$\mathbb{E}[T_i(n)]$ is the sum of the necessary pulls to resolve the gap and drift, plus the expected pulls due to random estimation errors. The necessary pulls are bounded by u plus N_0 . The pulls due to errors are bounded by summing the failure probabilities from steps $t = 1$ to n :

$$\sum_{t=1}^n \sum_{s=1}^t 2t^{-4} \leq \sum_{t=1}^{\infty} 2t^{-3} \leq \frac{\pi^2}{3} \quad (12)$$

We allocate N_0 samples to handle the initial transient drift for both the optimal and sub-optimal arms, accounting for the $2N_0$ term, $\frac{16C_p^2 \ln n}{(\Delta_i/2)^2}$ samples to resolve the gap Δ_i , and a constant term for the probabilistic noise:

$$\mathbb{E}[T_i(n)] \leq \frac{16C_p^2 \ln n}{(\Delta_i/2)^2} + 2N_0 + \frac{\pi^2}{3} \quad (13)$$

□

Theorem 3. Let $\bar{X}_n = \sum_{i=1}^K \frac{T_i(n)}{n} \bar{X}_{i,T_i(n)}$. Then:

$$|\mathbb{E}[\bar{X}_n] - \mu^*| \leq |\delta_n^*| + O\left(\frac{K(C_p^2 \ln n + N_0)}{n}\right) \quad (14)$$

where

$$\mu_{*,n} = \mu^* + \delta_n^* \implies \delta_n^* = \mu_{*,n} - \mu^*$$

Proof. We will bound the absolute difference between the expected empirical mean at the root, $\mathbb{E}[\bar{X}_n]$, and the true optimal value μ^* . $\bar{X}_n$ is the average of the rewards collected from all K arms over n . Let $T_i(n)$ be the number of times arm i is selected, and let $\bar{X}_{i,T_i(n)}$ be the average reward obtained from arm i . The estimator is given by:

$$\bar{X}_n = \sum_{i=1}^K \frac{T_i(n)}{n} \bar{X}_{i,T_i(n)} \quad (15)$$

By the linearity of expectation, we can express the expected value of this estimator as the sum of the expected cumulative rewards from each arm divided by n .

$$\mathbb{E}[\bar{X}_n] = \frac{1}{n} \left(\mathbb{E} \left[\sum_{t=1}^{T_*(n)} X_{*,t} \right] + \sum_{i \neq *} \mathbb{E} \left[\sum_{t=1}^{T_i(n)} X_{i,t} \right] \right) \quad (16)$$

For the optimal value μ^* , the constant μ^* as a weighted sum of itself using the identity $n = T_*(n) + \sum_{i \neq *} T_i(n)$:

$$\mu^* = \frac{1}{n} \left(\mathbb{E}[T_*(n)] \mu^* + \sum_{i \neq *} \mathbb{E}[T_i(n)] \mu^* \right) \quad (17)$$

Subtracting the expression for μ^* from $\mathbb{E}[\bar{X}_n]$. The bias can be bounded by the sum of the absolute errors contributed by the optimal arm and the suboptimal arms.

$$|\mathbb{E}[\bar{X}_n] - \mu^*| \leq \frac{1}{n} \left| \mathbb{E} \left[\sum_{t=1}^{T_*(n)} (X_{*,t} - \mu^*) \right] \right| + \frac{1}{n} \sum_{i \neq *} \left| \mathbb{E} \left[\sum_{t=1}^{T_i(n)} (X_{i,t} - \mu^*) \right] \right| \quad (18)$$

For the first term regarding the optimal arm. The non-stationary drift is defined such that the expected value of the average payoff deviates from the limit μ^* by δ_n^* . Specifically, for the optimal arm, the accumulated error is from the drift term scaled by the number of pulls. As $T_*(n) \leq n$ is bounded by $|\delta_n^*|$.

$$\frac{1}{n} \left| \mathbb{E} \left[\sum_{t=1}^{T_*(n)} (X_{*,t} - \mu^*) \right] \right| \leq |\delta_n^*| \quad (19)$$

For the second term regarding the suboptimal arms. Since rewards are bounded in $[0, 1]$, the maximum difference $|X_{i,t} - \mu^*|$ is at most 1. Therefore, the contribution of each suboptimal arm to the error is bounded by the fraction of times it was pulled:

$$\frac{1}{n} \sum_{i \neq *} \left| \mathbb{E} \left[\sum_{t=1}^{T_i(n)} (X_{i,t} - \mu^*) \right] \right| \leq \frac{1}{n} \sum_{i \neq *} \mathbb{E}[T_i(n)] \quad (20)$$

We now invoke (2), which provides a logarithmic bound on the expected number of pulls for suboptimal arms in UCT. $\mathbb{E}[T_i(n)] \leq \frac{16C_p^2 \ln n}{\Delta_i^2} + 2N_0 + \frac{\pi^2}{3}$. Summing this over all $K -$

1 suboptimal arms, the total number of suboptimal pulls grows as $O(K(C_p^2 \ln n + N_0))$. Substituting this back into the inequality yields:

$$|\mathbb{E}[\bar{X}_n] - \mu^*| \leq |\delta_n^*| + \frac{1}{n} O(K(C_p^2 \ln n + N_0)) \quad (21)$$

Then the bias converges to the drift of the optimal arm, with an additional error term that decays as $O(\frac{\ln n}{n})$. $\square$

Theorem 4 (Lower Bound). *There exists some positive constant ρ such that for all arms i and n :*

$$T_i(n) \geq \lceil \rho \log(n) \rceil$$

where constant ρ related to $(2C_p)^2$ such that:

Proof. For every arm i , the number of times it is selected, $T_i(n)$, grows at least logarithmically with the total number of trials n . We define the UCB index for any arm j at time n as:

$$I_j(n) = \bar{X}_{j,T_j(n)} + 2C_p \sqrt{\frac{\ln n}{T_j(n)}} \quad (22)$$

The algorithm selects the arm that maximizes this index: $I_{chosen}(n) = \max_j I_j(n)$. We assume there exists a dominant arm $*$ that is favored. The rewards are bounded in $[0, 1]$, and the empirical mean of any arm is at most 1. Even if the exploration bonus for arm $*$ is non-zero, as $n \rightarrow \infty$, if arm $*$ is pulled frequently ($T_*(n) \approx n$), its bonus term vanishes. Let us upper bound the index of any other arm j by a constant K_{max} , $K_{max} \approx 1 + \epsilon$. For arm i , the rewards are non-negative, so $\bar{X}_{i,T_i(n)} \geq 0$. The index for arm i is strictly lower bounded by its exploration bonus:

$$I_i(n) \geq 2C_p \sqrt{\frac{\ln n}{T_i(n)}} \quad (23)$$

Arm i will inevitably be selected if its lower bound index exceeds the upper bound index of all other arms, selection is guaranteed if:

$$2C_p \sqrt{\frac{\ln n}{T_i(n)}} \geq K_{max} \quad (24)$$

solve for $T_i(n)$:

$$\sqrt{\frac{\ln n}{T_i(n)}} \geq \frac{K_{max}}{2C_p} \quad (25)$$

Squaring both sides:

$$\frac{\ln n}{T_i(n)} \geq \left(\frac{K_{max}}{2C_p} \right)^2 \quad (26)$$

$$T_i(n) \leq \ln n \cdot \left(\frac{2C_p}{K_{max}} \right)^2 \quad (27)$$

It implies that if $T_i(n)$ were smaller than the Right side, the algorithm would be forced to pick arm i . Thus, to prevent the exploration bonus from dominating the decision process forever, the algorithm must maintain $T_i(n)$ large enough. If $T_i(n)$ falls below this threshold, the exploration term $2C_p\sqrt{\frac{\ln n}{T_i(n)}}$ becomes large enough to exceed the value of the optimal arm, triggering a selection of arm i and incrementing $T_i(n)$. $\square$

Theorem 5. Fix $\delta > 0$ and let $\Delta_n = 9\sqrt{2n \ln(2/\delta)}$. The following bounds hold true provided that n is sufficiently large:

$$\begin{aligned}\mathbb{P}(n\bar{X}_n \geq n\mathbb{E}[\bar{X}_n] + \Delta_n) &\leq \delta \\ \mathbb{P}(n\bar{X}_n \leq n\mathbb{E}[\bar{X}_n] - \Delta_n) &\leq \delta\end{aligned}$$

Proof. We construct a martingale difference sequence and apply the Azuma-Hoeffding inequality to bound the deviation of the cumulative reward from its expectation.

Let X_t is the reward received at time step t , where $X_t \in [0, 1]$. Let $\mathcal{F}_{t-1}$ be the filtration representing the history of actions and observations up to time $t-1$. Then the conditional expectation of the reward at time t is $\mu_t = \mathbb{E}[X_t \mid \mathcal{F}_{t-1}]$.

We construct a sequence of random variables Y_t as follows:

$$Y_t = X_t - \mu_t = X_t - \mathbb{E}[X_t \mid \mathcal{F}_{t-1}]. \quad (28)$$

$\mathbb{E}[Y_t \mid \mathcal{F}_{t-1}] = 0$, so $\{Y_t\}_{t=1}^n$ forms a martingale difference sequence and $Y_t \in [-1, 1]$.

Consider the total deviation S_n after n steps:

$$S_n = \sum_{t=1}^n Y_t = \sum_{t=1}^n X_t - \sum_{t=1}^n \mathbb{E}[X_t \mid \mathcal{F}_{t-1}]. \quad (29)$$

$n\bar{X}_n = \sum_{t=1}^n X_t$ and $n\mathbb{E}[\bar{X}_n]$ as the cumulative expected reward $\sum_{t=1}^n \mu_t$ we have $S_n = n\bar{X}_n - n\mathbb{E}[\bar{X}_n]$.

We now apply the Azuma-Hoeffding inequality. For a martingale difference sequence Y_t with bounded ranges $c_t = 1$, the inequality states that for any $\epsilon > 0$:

$$\mathbb{P}(|S_n| \geq \epsilon) \leq 2 \exp\left(-\frac{2\epsilon^2}{\sum_{t=1}^n c_t^2}\right) = 2 \exp\left(-\frac{2\epsilon^2}{n}\right). \quad (30)$$

We set the bound on the right-hand side to be equal to the confidence parameter δ :

$$2 \exp\left(-\frac{2\epsilon^2}{n}\right) = \delta. \quad (31)$$

Solving for ϵ :

$$\begin{aligned}\exp\left(-\frac{2\epsilon^2}{n}\right) &= \frac{\delta}{2} \\ -\frac{2\epsilon^2}{n} &= \ln\left(\frac{\delta}{2}\right) = -\ln\left(\frac{2}{\delta}\right) \\ \epsilon &= \sqrt{\frac{n}{2} \ln\left(\frac{2}{\delta}\right)}.\end{aligned}$$

Here ϵ represents the standard deviation bound. The theorem statement defines $\Delta_n = 9\sqrt{2n \ln(2/\delta)}$.

$$\Delta_n = 9 \cdot 2 \cdot \sqrt{\frac{n}{2} \ln \left(\frac{2}{\delta} \right)} = 18\epsilon. \quad (32)$$

The constant factor relaxes the non-stationary drift and facilitates the inductive proof in Theorem 7. Finally we have:

$$\mathbb{P}(|n\bar{X}_n - n\mathbb{E}[\bar{X}_n]| \geq \Delta_n) \leq \mathbb{P}(|n\bar{X}_n - n\mathbb{E}[\bar{X}_n]| \geq \epsilon) \leq \delta. \quad (33)$$

□

Theorem 6 (Convergence of Failure Probability). *Let $\hat{I}_t = \arg \max_i \bar{X}_{i,T_i(t)}$. Then:*

$$P(\hat{I}_t \neq i^*) \leq C \left(\frac{1}{t} \right)^{\frac{\rho}{2} \frac{\min_{i \neq i^*} \Delta_i^2}{36}}$$

with some constant C . In particular, it holds that:

$$\lim_{t \rightarrow \infty} P(\hat{I}_t \neq i^*) = 0$$

Proof. We will bound the probability that the algorithm selects a suboptimal arm i instead of the optimal arm $*$ after t episodes. Let $\hat{I}_t$ be the arm with the highest empirical mean at time t . The event $\hat{I}_t = i$ implies that the empirical mean of the suboptimal arm exceeds that of the optimal arm:

$$\bar{X}_{i,T_i(t)} \geq \bar{X}_{*,T_*(t)} \quad (34)$$

Let μ_i and μ^* be the true converged values of the suboptimal and optimal arms, with $\Delta_i = \mu^* - \mu_i > 0$. Then the combined estimation error of both arms must exceed the gap Δ_i . We must have at least one of the following deviations.

1. The optimal arm is underestimated by at least $\Delta_i/2$: $\bar{X}_{*,T_*(t)} \leq \mu^* - \Delta_i/2$.
2. The suboptimal arm is overestimated by at least $\Delta_i/2$: $\bar{X}_{i,T_i(t)} \geq \mu_i + \Delta_i/2$.

We apply the Hoeffding inequality to bound these probabilities. The probability of a deviation of size ϵ with N samples decays as $\exp(-2N\epsilon^2)$. For the suboptimal arm i , from Theorem 4 we know that UCT guarantees a logarithmic lower bound on the number of samples for every arm:

$$T_i(t) \geq \lceil \rho \ln t \rceil \quad (35)$$

The optimal arm $*$ is pulled even more frequently, $T_*(t) \approx t$, so its error probability decays exponentially in t and is negligible compared to the suboptimal arm. The failure probability is dominated by the limited sampling of the suboptimal arm i .

Substituting the lower bound $N = \rho \ln t$ and the deviation $\epsilon \propto \Delta_i$ into the exponential decay bound:

$$P(\text{Error}_i) \leq \exp \left(-2T_i(t) \left(\frac{\Delta_i}{C_{scale}} \right)^2 \right) \quad (36)$$

where C_{scale} is a constant accounting for the drift handling. Substituting $T_i(t) \geq \rho \ln t$:

$$P(\text{Error}_i) \leq \exp \left(-2(\rho \ln t) \left(\frac{\Delta_i}{36} \right)^2 \right) \quad (37)$$

Using the identity $a \ln x = \ln(x^a)$ and $\exp(-\ln y) = 1/y$:

$$P(\text{Error}_i) \leq \exp \left(-\ln \left(t^{2\rho(\Delta_i/36)^2} \right) \right) = \left(\frac{1}{t} \right)^{2\rho(\frac{\Delta_i}{36})^2} \quad (38)$$

Summing over all suboptimal arms, the total failure probability is bounded by the sum of these polynomial decay terms. Since $\lim_{t \rightarrow \infty} t^{-\alpha} = 0$ for any $\alpha > 0$, the failure probability converges to zero.

$$\lim_{t \rightarrow \infty} P(\hat{I}_t \neq i^*) = 0 \quad (39)$$

This confirms that UCT is a consistent estimator of the best action. $\square$

Theorem 7. *Consider a finite-horizon MDP with rewards scaled to lie in the $[0, 1]$ interval. Let the horizon of the MDP be D , and the number of actions per state be K . Consider algorithm UCT such that the bias terms of UCB1 are multiplied by D . Then the bias of the estimated expected payoff, $\bar{X}_n$, is $O(\log(n)/n)$. Further, the failure probability at the root converges to zero at a polynomial rate as the number of episodes grows to infinity.*

Proof. We do induction on the depth of the search tree, denoted by D . **Base Case ($D = 1$):** The leaf nodes return immediate rewards from the MDP, which are stationary distributions; the drift term δ_n^* is zero. Applying Theorem 3 with $\delta_n^* = 0$:

$$|\mathbb{E}[\bar{X}_n] - \mu^*| \leq 0 + O \left(\frac{K \log n}{n} \right) = O \left(\frac{\log n}{n} \right) \quad (40)$$

the tail conditions required for UCB1 Theorem 5 are satisfied with the standard constant $C_p = 1/\sqrt{2}$ by Hoeffding's inequality. The failure probability converges to zero. Thus, the theorem holds for $D = 1$.

Inductive Step: Assume that the theorem holds for all trees of depth up to $D - 1$. We now consider a tree of depth D . We scale all cumulative rewards by dividing by D , mapping them to the interval $[0, 1]$.

Consider the root node of this depth- D tree. When the algorithm selects an arm i , it transitions to a child node which is the root of a subtree of depth $D - 1$. Let $X_{i,t}$ be the reward sequence returned by this child node. This sequence is non-stationary because the child node is running UCT and updating its own estimates. However, the bias of the child node's estimate converges at a rate of $O(\frac{\log t}{t})$. This implies that the drift of the payoff distribution experienced by the root, δ_t , is bounded by this same rate:

$$|\delta_t| = O \left(\frac{\log t}{t} \right) \quad (41)$$

This drift vanishes as $t \rightarrow \infty$. We must now verify that the non-stationary sequence $X_{i,t}$ satisfies the tail conditions (drift conditions) required to apply Theorems 1 and 2 at the

root. Since the child nodes (depth $D - 1$) satisfy the convergence properties by hypothesis, Theorem 5 guarantees that the aggregated rewards at the root also satisfy the necessary concentration inequalities and tails decay exponentially.

We can now apply Theorem 3 to the root node. The total bias is bounded by the drift of the optimal child plus the regret from the root’s own exploration:

$$|\mathbb{E}[\bar{X}_n] - \mu^*| \leq |\delta_n^*| + O\left(\frac{K \log n}{n}\right) \quad (42)$$

Substituting the bound for the optimal child’s drift (from the inductive hypothesis):

$$|\mathbb{E}[\bar{X}_n] - \mu^*| \leq O\left(\frac{\log n}{n}\right) + O\left(\frac{\log n}{n}\right) \quad (43)$$

The sum of two $O(\frac{\log n}{n})$ terms is still $O(\frac{\log n}{n})$. Thus, the bias at the root of a depth- D tree converges at the required rate.

Since the bias vanishes and exploration ensures infinite sampling Theorem 4, the conditions in Theorem 6 are met. The probability that the root node selects a suboptimal arm converges to zero at a polynomial rate. $\square$

8 Experiment

To empirically validate the convergence properties of the PO-UCT algorithm as described in Theorem 1, we conducted two distinct experiments using the rocksample problem, since that is a typical problem and generally not intractable. Both experiments were run on a RockSample($n=6, k=7$) environment. The key parameters used for planning were: Discount Factor $\gamma = 0.95$ Exploration Constant $C = 5$. This depends on the problem setting, and a better chosen C will make the convergence faster. Rollout Policy is just a simple random policy. Belief Representation is a particle filter with 500 particles, initialized with a uniform belief of rock qualities.

8.1 Convergence of the Value Estimate $V(h)$

This experiment tests the theoretical convergence of the value function $V(h)$ at a fixed point. We repeatedly estimated the value of the initial history, that is, the start point $h_0 = h_initial$, using a varying number of simulations (N). For each N value in our test range (from $N = 100$ to $N = 300,000$), we ran the planner $M = 200$ times on the same fixed problem instance and computed the mean and variance of the resulting $V(h)$ estimates.

8.1.1 Mean Value Convergence

The results for the mean estimated value, $\mathbb{E}[V(h)]$, are shown in Figure 2. and Figure 3 plots the variance of the $V(h)$ estimates from the same experiment.

V(h) Convergence (Mean) at h='h_initial' (M=200 trials, n=6, k=7)

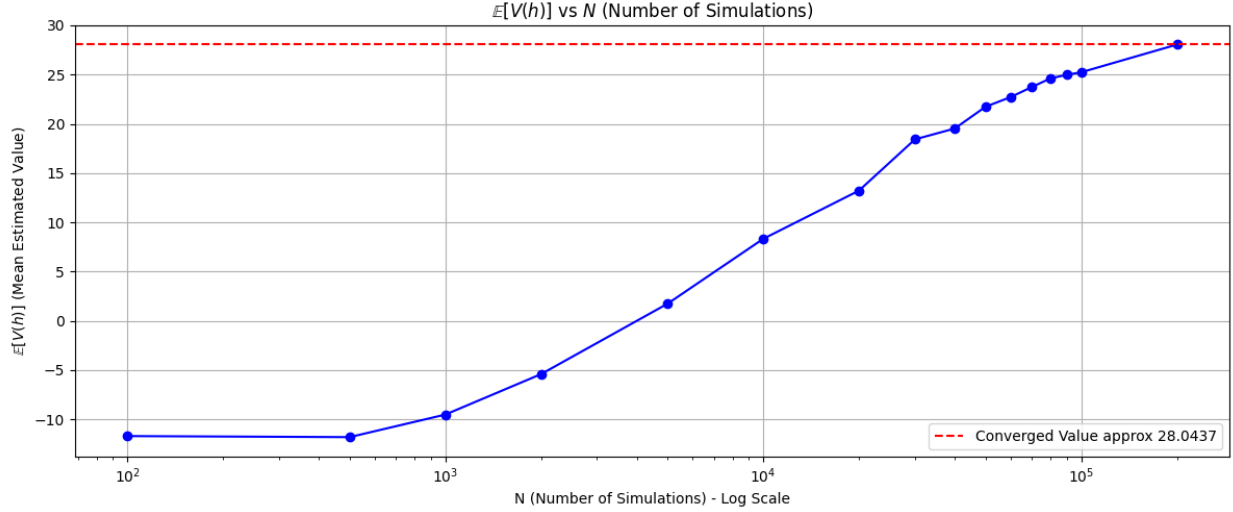


Figure 2: Mean value function $\mathbb{E}[V(h)]$ convergence at $h = 'h_initial'$. The plot shows the average estimated value (from $M = 200$ trials) as a function of the number of simulations (N) per planning step for the RockSample(6,7) problem. The value estimate (blue line) steadily increases from a negative estimate at low N and converges to an asymptotic value as N increases. This strongly supports the convergence property of the POMCP algorithm.

8.1.2 Variance Converges Zero

Theoretically, the bias $O(\log N(h)/N(h))$ implies the variance should always decrease as $N \rightarrow \infty$. However, our results show a convexity in variance. This is not a failure of the theorem, but highlights a practical limitation: the variance of the random rollout policy is high. As N increases, the search tree expands into new, highly uncertain regions where the random rollout provides inconsistent reward estimates, thus increasing the overall sample variance. As N increases, the search explores deeper, and the high variance of the random rollouts begins to dominate the statistics, causing the sample variance of $V(h)$ to increase before it can begin its theoretical convergence to zero.

8.2 Convergence of Policy Quality

The second experiment validates the practical implication of Theorem 1: that a better value estimate higher N leads to a better real-world policy. We ran $M = 200$ complete, independent episodes for the RockSample(6,7) problem. Each episode was run with a fixed number of simulations N per decision step. Figure 4 shows the distribution of the final total discounted rewards.

This demonstrates the practical convergence of the algorithm. Increasing the simulation yields better policies, up to a point of diminishing returns. Until the policy is good enough, and further computation does not significantly improve the outcome.

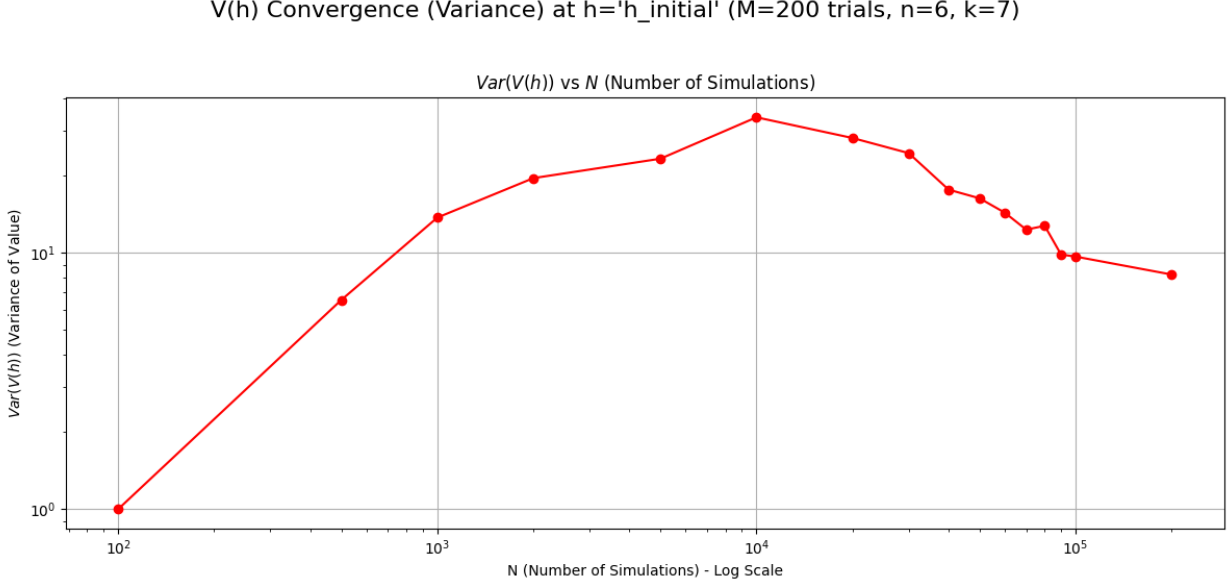


Figure 3: Variance of the value function $Var(V(h))$ at $h = 'h_initial'$. The plot shows the sample variance of the $V(h)$ estimate (across $M = 200$ trials) as a function of N . For N large enough, it starts a slow decline.

9 Future work and Discussion

A promising direction for future research is the application of online POMDP planning algorithms to accelerate Markov Chain Monte Carlo (MCMC) sampling in complex, high-dimensional generative models. The MCMC environment is the target probability distribution $p(x)$ that we aim to sample from. This distribution is often defined implicitly through an unnormalized energy function $E(x)$, where the probability of a state x is given by the Boltzmann distribution:

$$p(x) = \frac{1}{Z} \exp(-E(x))$$

$Z = \sum_x \exp(-E(x))$ is the partition function, which is computationally intractable to compute. The environment $p(x)$ is static.

9.1 The MCMC Sampling Problem as a POMDP

In generative models, MCMC sampling is often computationally expensive and suffers from slow mixing times, especially in multi-modal distributions (distributions with multiple, separated high-probability regions). This is largely because standard MCMC methods use fixed or constructed proposal distributions (random Gaussian noise), which explore the state space inefficiently.

We conjecture that this sampling process can be reformulated as an online planning problem, effectively an MDP or POMDP:

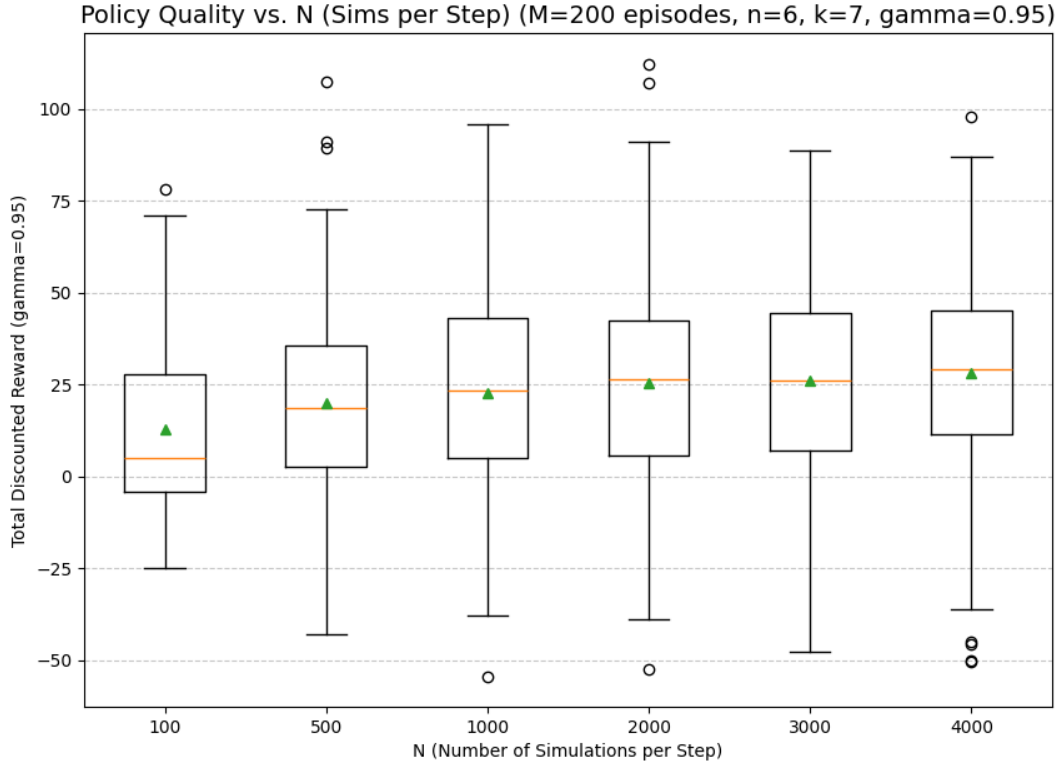


Figure 4: Policy Quality vs. N (Simulations per Step). This box plot shows the distribution of final total discounted rewards (Y-axis) obtained over $M = 200$ independent episodes for the RockSample(6,7) problem. Each box corresponds to a different N on the X-axis. The mean (green triangle) and median (orange line) reward show an improvement when N is increased from 100 to 500, and again from 500 to 1000. Performance then saturates for $N \geq 1000$.

- **State (s):** The current sample in the chain x_t .
- **Action (a):** The *choice* of a proposal mechanism from a predefined set $\mathcal{A}$. For example, $\mathcal{A} = \{\text{local perturbation, global jump, gradient-based proposal}\}$.
- **History (h):** The sequence of past samples, $h_t = (x_0, \dots, x_t)$.
- **Policy ($\pi(a|h)$):** A learned policy to optimize the sampling process. Given the history, intelligently selects the most promising proposal action (a_{t+1}) to explore the state space efficiently, rather than using a fixed proposal.
- **Reward (R):** This is an artificial setu we would design. The reward function $R(h_t, a, x_{t+1})$ must be engineered to encourage faster mixing and more effective exploration.

- Provide a high reward for generating a sample with a high acceptance probability and for generating a sample x_{t+1} that has low autocorrelation with recent samples since it successfully explored a new part of the space.
- Provide a negative reward (penalty) if the chain remains stuck in the same region for too long, we force the agent to jump out of the valley.

9.2 Accelerating Sampling via Online Planning

Instead of a fixed MCMC transition rule, an online planner like POMCP using the UCT formula could be employed at each sampling step t . Given the current state x_t and history h_t , the planner would execute N simulations to determine the optimal action $a^* \in \mathcal{A}$.

The planner’s simulations would use the artificial reward function R to estimate the long-term value $Q(h, a)$ of choosing a particular proposal mechanism. For instance, the planner might learn through simulation. Assume that the last bunch of samples were highly correlated, that is bad history h , the estimated value $Q(h, \text{local perturbation})$ is low. However, the value $Q(h, \text{global jump})$ is high, as its simulations show a greater probability of discovering a new distribution mode and receiving a higher future reward. This intelligent sampler can learn to navigate energy landscapes theoretically, much more efficiently than a blind random walk, reducing the total number of iterations (MCMC steps) required for the chain to converge and mix.

9.3 Challenges of Computational Trade-offs

This approach requires a significant computational trade-off between convergence and simulation costs. While the total number of MCMC iterations required for convergence may decrease, as we hope, the computational cost per iteration increases a lot. Each step now requires a full MCTS planning phase to select the next action.

The viability of this method lies on whether the reduction in required steps is drastic enough to offset the increased cost-per-step. Furthermore, the primary research challenge lies in designing a reward function R that is tractable, as discussed before.

10 Conclusions

In this paper, we present an overall analysis of the Partially Observable Monte Carlo Planning (POMCP) algorithm, demonstrating its efficacy in solving the high-dimensional and computationally challenging POMDP. We establish a formal basis for its application and then use a particle filter to manage and update the belief space. Our primary theoretical work is the rigorous proof of convergence. By demonstrating that the POMDP is mathematically equivalent to a derived, history-based MDP. We showed that the PO-UCT search at the core of POMCP inherits the well-established convergence guarantees of standard UCT, ensuring that the value estimates converge to the optimal value V^* as the simulation time increases. This also suggests a practical bottleneck: the high variance introduced by the simple

random rollout policy (Figure 3) which will slow the speed of convergence. This finding reminds us that a future improvement may lie in the design of domain-specific heuristic rollout policies to enhance sample efficiency. P as a provably convergent online planning framework for large-scale POMDPs. As discussed in our future work, the principles of online policy selection explored here may be applied to accelerate other complex computational tasks, such as MCMC sampling.

References

- [1] Karl J Åström. Optimal control of Markov processes with incomplete state information. *Journal of Mathematical Analysis and Applications*, 10(1):174–205, 1965.
- [2] Richard Bellman. A Markovian decision process. *Journal of Mathematics and Mechanics*, 6(4):679–684, 1957.
- [3] M Böger, B De Saporta, S Eyheramendy, and S Panche. A pomdp formulation of the non-destructive testing problem. In *Prognostics and Health Management Conference (PHM-E)*. IEEE, 2021.
- [4] Iadine Chadès, Luz V. Pascal, Sam Nicol, Cameron S. Fletcher, and Jonathan Ferrer-Mestres. A primer on partially observable markov decision processes (pomdps). *Methods in Ecology and Evolution*, 12(11):2058–2072, 2021. doi: 10.1111/2041-210X.13692.
- [5] Rémi Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *5th International Conference on Computer and Games*, pages 72–83, 2006.
- [6] Edgar Galván and Gavin Simpson. On the evolution of the mcts upper confidence bounds for trees by means of evolutionary algorithms in the game of carcassonne. *arXiv preprint arXiv:2112.09697*, 2021.
- [7] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- [8] L. Kocsis and C. Szepesvári. Bandit based monte-carlo planning. In *15th European Conference on Machine Learning*, pages 282–293, 2006.
- [9] Hanna Kurniawati, David Hsu, and Wee Sun Lee. SARSOP: Efficient point-based POMDP planning by approximating optimally reachable belief spaces. In *Robotics: Science and Systems*, 2008.
- [10] Jongmin Lee, Geon-Hyeong Kim, Pascal Poupart, and Kee-Eung Kim. Monte-carlo tree search for constrained POMDPs. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, Montréal, Canada, 2018.
- [11] Michael H. Lim, Tyler J. Becker, Mykel J. Kochenderfer, Claire J. Tomlin, and Zachary N. Sunberg. Optimality guarantees for particle belief approximation of POMDPs. *Journal of Artificial Intelligence Research*, 77:1591–1636, 8 2023.

- [12] Giulio Mazzi, Alberto Castellini, and Alessandro Farinelli. Rule-based shielding for partially observable monte-carlo planning. In *Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling (ICAPS 2021)*, pages 243–251, 2021.
- [13] Giulio Mazzi, Alberto Castellini, and Alessandro Farinelli. Risk-aware shielding of partially observable monte carlo planning policies. *Artificial Intelligence*, 324:103987, 2023. doi: 10.1016/j.artint.2023.103987.
- [14] Christos H Papadimitriou and John N Tsitsiklis. The complexity of Markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987.
- [15] Joelle Pineau, Geoff Gordon, and Sebastian Thrun. Anytime point-based approximations for large POMDPs. *Journal of Artificial Intelligence Research*, 27:335–380, 2006.
- [16] Martin L Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2014.
- [17] Stéphane Ross, Joelle Pineau, Sylvie Paquet, and Brahim Chaib-draa. Online planning algorithms for POMDPs. *Journal of Artificial Intelligence Research*, 32:663–704, 2008.
- [18] Alan L Schaefer, Mark S Roberts, Theresa K Smith, Andrew G D’Souza, Stephen K Mike, Gillian D Sanders, and Douglas K Owens. Optimizing sequential medical decisions: A pomdp approach. *Operations Research*, 53(2):229–243, 2005.
- [19] Dhruv Shah, Błażej Osiński, Brian Ichter, and Sergey Levine. LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action. *arXiv preprint arXiv:2207.04429*, 2022.
- [20] David Silver and Joel Veness. Monte-carlo planning in large pomdps. In *Advances in Neural Information Processing Systems 23*, pages 2164–2172, 2010.
- [21] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [22] Trey Smith and Reid Simmons. Heuristic search value iteration for pomdps. In *Proceedings of the Twentieth conference on Uncertainty in artificial intelligence (UAI)*, pages 520–527, 2004.
- [23] Edward J Sondik. *The optimal control of partially observable Markov processes*. PhD thesis, Stanford University, 1971.
- [24] Matthijs T.J. Spaan. Partially observable markov decision processes. In M. A. Wiering and M. van Otterlo, editors, *Reinforcement Learning: State of the Art*. Springer Verlag, 2012.
- [25] Lauren N. Steimle and Brian T. Denton. Markov decision processes for screening and treatment of chronic diseases. In Richard J. Boucherie and Nico M. van Dijk, editors, *Markov Decision Processes in Practice*, International Series in Operations Research

- & Management Science, pages 189–222. Springer International Publishing, 2017. doi: 10.1007/978-3-319-47766-4_6.
- [26] The JuliaPOMDP Developers. POMDPs.jl gallery. <https://juliapomdp.github.io/POMDPs.jl/stable/gallery/>, 2025. Accessed: 2025-11-13.
 - [27] The JuliaPOMDP Developers. Example pomdps • particlefilters.jl. <https://juliapomdp.github.io/ParticleFilters.jl/latest/example-pomdps/>, 2025. Accessed: 2025-11-13.
 - [28] Jason D Williams and Steve Young. Partially observable Markov decision processes for spoken dialog systems. *Computer Speech & Language*, 21(2):393–422, 2007.
 - [29] Feng Wu, Shlomo Zilberstein, and Xiaoping Chen. Rollout sampling policy iteration for decentralized pomdps. In *Proceedings of the 26th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 666–673, Catalina Island, CA, USA, 2010. AUAI Press.
 - [30] Steve Young, Milica Gašić, Blaise Thomson, and Jason D. Williams. POMDP-based Statistical Spoken Dialogue Systems: a Review. *Proceedings of the IEEE*, 101(5):1163–1183, May 2013.